



Multimodal Representation Learning for Binary Code Similarity Analysis: A Systematic Review and Conceptual Framework

Rupesh Kohli^{*}, Harish Parshuram Bhabad, Atmeshkumar Subhashbhai Patel, Vijay M. Rakhade, Nandini S. Patil, Vedant Kadlag

Department of Artificial Intelligence and Data Science, Sandip Institute of Technology and Research Centre (SITRC), 422213 Nashik, India

* Correspondence: Rupesh Kohli (rupeshkohli.aids@sitrc.org)

Received: 05-08-2026

Revised: 06-13-2026

Accepted: 06-22-2026

Citation: R. Kohli, H. P. Bhabad, A. S. Patel, V. M. Rakhade, N. S. Patil, and V. Kadlag, "Multimodal representation learning for binary code similarity analysis: A systematic review and conceptual framework," *Inf. Dyn. Appl.*, vol. 5, no. 2, pp. 110–129, 2026. <https://doi.org/10.56578/ida050204>.



© 2026 by the author(s). Published by Acadlore Publishing Services Limited, Hong Kong. This article is available for free download and can be reused and cited, provided that the original published version is credited, under the CC BY 4.0 license.

Abstract: Binary code similarity analysis is essential for software reverse engineering, vulnerability discovery, and malware analysis. However, conventional unimodal representations relying exclusively on linear opcode streams, control-flow graph (CFG) topologies, or dynamic system-call traces lack robustness when confronted with compiler transformations, adversarial obfuscation (e.g., Ultimate Packer for eXecutables (UPX) packing, control-flow flattening), and anti-analysis evasion. This study addresses these limitations through a systematic literature review and a unified conceptual framework. Following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 guidelines, we systematically search five major academic digital databases (IEEE Xplore, ACM Digital Library, ScienceDirect, Scopus, and SpringerLink) covering 2020 through 2026. From 5,650 initially identified records, 11 primary benchmark and foundational studies are retained and categorized as direct multimodal or component-level evidence under an operational eight-dimension quality rubric (Q1–Q8). Based on the evidence synthesis, a conceptual architecture termed Multi-Modal Contrastive Binary Similarity Analysis (MM-CBSA) was proposed. Opcode sequences, control-flow graphs, and system-call traces are encoded using Transformer, Graph Isomorphism Network (GIN), and Bidirectional Long Short-Term Memory (BiLSTM) encoders, respectively, and projected onto a shared unit hypersphere. Cross-modal alignment is achieved via multi-channel Information Noise-Contrastive Estimation (InfoNCE) objectives with volumetric Gram regularization, paired with reliability-aware dynamic gating to accommodate degraded or missing modalities. Technical feasibility was examined using an associated open-source reference implementation. On a stored 1,500-sample test artifact (744 malware, 756 benign), the implementation achieved preliminary classification performance of 99.73% accuracy and 0.9973 F1-score, with a mean neural forward-pass latency of 28.87 ms on Central Processing Unit (CPU). Under preliminary stress testing, accuracy decreased to 88.0% under UPX packing and to 80.0% under dead-code insertion. Crucially, the manuscript establishes that current empirical evidence supports binary classification rather than direct binary similarity retrieval, and stored temporal and family-holdout artifacts warrant independent experimental revalidation. These observations provide technical-feasibility evidence only and do not constitute direct validation of binary code similarity retrieval. Accordingly, a reproducible empirical validation protocol is formulated to support future evaluation of multimodal binary code similarity analysis under compiler variation, obfuscation, distribution shift, and modality degradation.

Keywords: Binary code similarity analysis; Multimodal representation learning; Control-flow graph; Graph isomorphism networks; Dynamic analysis

1 Introduction

Binary code analysis is a cornerstone of modern software security, providing essential analytical capabilities for vulnerability triage, patch verification, software intellectual property auditing, and automated malware identification [1, 2]. When analyzing stripped executables in which debug symbols, source-code comments, type signatures, and variable identifiers have been permanently removed analysts must deduce underlying program semantics, control intent, and architectural functionality directly from raw machine instructions [3]. Over the past decade, automated

learning-based binary analysis has advanced rapidly, transitioning from handcrafted syntactic heuristics to deep neural representations capable of projecting binary code fragments into dense vector spaces [4, 5].

Despite these algorithmic advances, conventional binary representation learning systems remain fundamentally constrained by their reliance on single-modality observation channels [6, 7]. A compiled binary executable is an intrinsically multifaceted software artifact that can be observed through three primary representations: (a) linear assembly opcode streams (X_O), which preserve sequential instruction semantics, register allocations, and immediate operations; (b) control-flow graphs (G_C), which encode the non-Euclidean topological branching structures and inter-block execution logic; and (c) dynamic execution traces (S_D), which record the chronological sequence of system calls, Application Programming Interface (API) parameters, and kernel interactions generated during sandbox execution [8–10]. As demonstrated across extensive reverse-engineering literature, each unimodal representation exhibits structural vulnerabilities when subjected to standard compiler optimizations and adversarial evasion techniques [11–15]. The complementary characteristics and limitations of these three representation views are summarized in Figure 1.

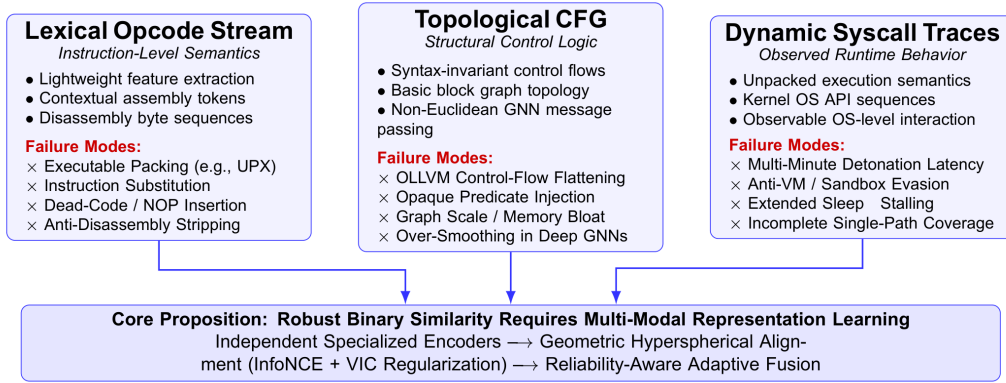


Figure 1. Conceptual overview of three primary representation views in binary analysis and their associated failure modes

Note: API = Application Programming Interface; BCSA = Binary Code Similarity Analysis; CFG = Control-Flow Graph; GNN = Graph Neural Network; InfoNCE = Information Noise-Contrastive Estimation; UPX = Ultimate Packer for eXecutables.

Specifically, linear opcode sequence models (e.g., recurrent architectures and instruction Transformers) capture fine-grained arithmetic and data movement operations, but they are highly fragile against compiler optimization levels (e.g., -O0 versus -O3), instruction reordering, dead-code insertion, and register renaming [16, 17]. Structural graph representations process basic-block control-flow graphs using Graph Neural Networks (GNNs) to achieve invariance to linear syntactic reordering [4, 7, 18, 19]; however, they remain bounded by the expressiveness limits of graph isomorphism testing [5] and are severely degraded by control-flow flattening, bogus branch insertion, and opaque predicates [7, 20]. Conversely, dynamic system-call execution profiling monitors concrete kernel interactions in instrumented sandboxes, providing resilience against static packing and code encryption [8–10]. Nonetheless, dynamic profiling incurs heavy computational latency (often requiring tens of seconds per binary), suffers from incomplete execution path coverage, and is easily foiled by environment detection routines, sleep stalling, and anti-sandbox evasion [8, 9, 21].

The presence of these orthogonal failure modes establishes a compelling motivation for multimodal representation learning [14, 15]. By jointly projecting opcode sequences, control-flow topologies, and dynamic execution traces into a unified latent space, a multimodal framework can achieve semantic complementarity, wherein the analytical strengths of one modality compensate for the degradation or absence of another [11, 14]. However, existing binary analysis research suffers from four critical methodological gaps: (1) cross-modal representations are typically fused through naive concatenation or unaligned pooling without geometric alignment; (2) current architectures lack dynamic mechanisms to withstand missing, truncated, or actively corrupted modality channels; (3) published benchmarks frequently conflate binary classification with direct binary similarity retrieval; and (4) experimental evaluations are often compromised by temporal data leakage and artificial benchmark regularities.

To address these challenges, this study formulates four central research questions:

- RQ1 (Modality Representation & Encoders): What neural representations and deep encoder architectures are most effective for capturing fine-grained instruction syntax, topological control flow, and runtime kernel interactions from stripped binary executables?

- RQ2 (Cross-Modal Alignment & Reliability Fusion): How can disparate representation channels be geometrically aligned in a shared latent hypersphere and adaptively weighted to withstand missing, uninformative, or adversarially corrupted modalities?

- RQ3 (Adversarial Robustness & Modality Complementarity): What theoretical and empirical evidence exists in the literature and preliminary reference implementations regarding multimodal resilience against compiler transformations and adversarial perturbations (e.g., UPX packing, dead-code insertion, control-flow obfuscation, sandbox stalling)?

- RQ4 (Benchmarking Gaps, Leakage Controls & Generalization): What methodological limitations affect existing binary analysis benchmarks, and what verification protocols are required to evaluate true binary similarity retrieval, temporal stability, and out-of-distribution generalization?

To answer these research questions, this paper delivers three primary scientific contributions:

1. Systematic Review and Evidence Synthesis: Following the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 protocol [22], we conduct a systematic review across five major academic digital libraries from 2020 through 2026, evaluating 5,650 candidate records and synthesizing 11 primary benchmark and foundational evidence studies under an operational eight-dimension quality rubric (Q1–Q8).

2. Conceptual Multi-Modal Contrastive Binary Similarity Analysis (MM-CBSA) Framework: We formalize the MM-CBSA framework, unifying opcode Transformers, CFG Graph Isomorphism Networks (GINs), and system-call BiLSTMs into a shared unit hypersphere via multi-channel InfoNCE alignment, volumetric Gram regularization, and reliability-aware dynamic gating.

3. Preliminary Technical-Feasibility Evidence and Methodological Audit: Examining an open-source experimental reference implementation on a stored 1,500-sample benchmark test artifact (744 malware, 756 benign), we report preliminary classification performance (99.73% accuracy, 0.9973 F1, 28.87 ms forward latency) and quantify preliminary degradation patterns under UPX packing (88.0% accuracy) and dead-code insertion (80.0% accuracy, 40.0% FNR, 0.0% FPR). Crucially, we transparently demarcate preliminary classification observations from unvalidated binary similarity retrieval and establish a standardized benchmark verification protocol.

4. Scientific Positioning, Scope, and Claim Discipline: In accordance with editorial guidance, this study is positioned primarily as a systematic literature review and conceptual framework, with the experimental findings serving as preliminary supporting evidence of technical feasibility. We explicitly maintain a strict methodological distinction between Binary Code Similarity Analysis (BCSA) and binary malware classification. BCSA is fundamentally an information retrieval and clone search problem ranking semantically equivalent functions or binaries across compiler, optimization, and architecture variations evaluated via Recall@K, Mean Reciprocal Rank (MRR), and Mean Average Precision (MAP). Binary malware classification, in contrast, evaluates coarse-grained binary categorical prediction (benign versus malicious). The empirical results reported in this paper reflect preliminary classification feasibility on a stored synthetic test artifact and do not constitute direct experimental validation of BCSA retrieval. Stored temporal and family-holdout artifacts produced identical aggregate metrics, indicating benchmark regularity rather than confirmed out-of-distribution generalization. All claims throughout this manuscript are rigorously calibrated to verified evidence.

2 Related Work and Literature Evidence

2.1 Instruction and Opcode Sequence Representations

Early learning-based binary analysis models represented disassembled code as bag-of-words or n-gram distributions over mnemonic tokens [1]. To capture contextual semantics, Ullah and Oh [23] introduced BinDiffNN, adapting distributed representations of assembly instructions to model syntactic and semantic variations in binary diffing. Kim et al. [19] revisited binary code similarity analysis using interpretable feature engineering, demonstrating the importance of normalized instruction semantics. More recently, Amouei et al. [24] developed LENA, leveraging large Transformer language models to generate contextual embeddings of neutralized assembly instructions across compiler optimization levels. Jiang et al. [25] introduced BinCola, proposing diversity-sensitive contrastive learning to enhance semantic differentiation across compiled assembly sequences. In parallel, Ito et al. [26] employed neural machine translation architectures to extract cross-architecture feature representations from binary instruction streams. While sequence models capture fine-grained register operations and intra-block semantics, their primary vulnerability remains their reliance on linear order: adversarial dead-code insertion, instruction swapping, and basic-block shuffling degrade sequence similarity even when core program logic is preserved [18, 20].

2.2 Control-Flow Graph and Structural Representations

To achieve invariance to linear syntax modifications, graph-based approaches model binaries as Control-Flow Graphs (CFGs), where vertices represent basic blocks and directed edges represent jump or branch transitions [2, 3]. Alrabaee et al. [2, 3] pioneered semantic flow graph and graph matching approaches (BinGold and SIGMA) to identify reused functions and structural semantics across compiled binaries. Luo et al. [7] developed semantics-based obfuscation-resilient comparison using symbolic execution and constraint solving over basic blocks. Feng et al. [4] introduced BejaGNN, applying GNNs to interprocedural CFGs for behavioral Java malware identification. Despite these strengths, Wu et al. [5] demonstrated that conventional message-passing GNNs suffer from theoretical

expressiveness limits bounded by the 1-Weisfeiler-Lehman (1-WL) graph isomorphism test, and remain vulnerable to control-flow flattening, bogus branch insertion, and opaque predicates [7, 20].

2.3 Dynamic Behavioral and System-Call Representations

Dynamic analysis captures the interactions between an untrusted binary and the operating system kernel by monitoring API calls, registry operations, network sockets, and file-system activity during execution in a guest sandbox [8, 10]. Early dynamic models applied Markov chains or sequential neural architectures to linear system-call logs [9, 10]. Amer and Zelinka [9] and Amer et al. [10] advanced dynamic behavioral modeling through contextual and multi-perspective fusion of API call sequences in Windows executables. Li et al. [11] introduced deep feature extractors for intrinsic API transition dynamics, while Haq et al. [12] and Amer et al. [6] developed graph-based dynamic architectures (GraphShield) to model system-call topologies. However, dynamic profiling imposes heavy computational overhead (typically requiring 30–120 seconds of sandbox observation per sample), exhibits incomplete execution path coverage, and fails completely against sophisticated malware employing environment-detection routines, anti-VM checks, or deliberate sleep delays [8, 9, 21].

2.4 Multimodal Representation Learning and Contrastive Frameworks

Recognizing the complementary strengths of static and dynamic modalities, recent research has explored multimodal fusion [10, 15]. Amer et al. [10] combined multi-perspective behavioral views, while Mahouachi [15] demonstrated knowledge distillation across multimodal representations for software vulnerability identification. Davies et al. [13] and Li et al. [14] integrated structural and semantic indicators for binary-level attack detection. By jointly embedding complementary views, multimodal architectures substantially reduce the individual blind spots of unimodal representations.

2.5 Literature Gap Analysis

Our systematic analysis of the literature reveals four critical gaps that motivate the proposed MM-CBSA framework: (1) existing multimodal models rely primarily on early concatenation or late score voting, failing to establish geometric cross-modal alignment in latent space; (2) current systems assume all modalities are available and trustworthy at inference time, lacking dynamic mechanisms to withstand missing or adversarial channels; (3) published literature frequently reports high accuracy on binary classification tasks while claiming broad binary similarity analysis capabilities, without evaluating direct retrieval metrics (e.g., Recall@K, Mean Reciprocal Rank); and (4) evaluations on synthetic or temporally unconstrained benchmarks exhibit inflated performance due to benchmark leakage, masking severe degradation in real-world triage environments under concept drift, as thoroughly documented by Sabbah et al. [27], Patel et al. [28], and Apruzzese et al. [29]. Recent hybrid architectures such as Codeformer [30] have attempted to bridge structural and sequence views, but lack dynamic reliability-aware gating against active adversarial perturbations. A detailed comparative synthesis of these surveyed paradigms and their operational quality assessments is presented in Section 3.3.

3 Systematic Literature Review Methodology

To establish a comprehensive and reproducible empirical baseline for multimodal binary analysis, this study conducted a systematic literature review in rigorous compliance with the PRISMA 2020 guidelines [22]. The review protocol was pre-formulated to identify, screen, and synthesize modern research published between January 2020 and February 2026 across five authoritative digital indexing platforms: IEEE Xplore, ACM Digital Library, ScienceDirect (Elsevier), Scopus, and SpringerLink. The overall study selection process following the PRISMA 2020 framework is illustrated in Figure 2.

3.1 Search Strategy and Digital Database Query Formulations

Search strings were constructed by combining controlled vocabulary terms with free-text descriptors across four conceptual facets: (a) representation targets, including “binary code”, “stripped binary”, “assembly”, and “machine code”; (b) modality types, including “opcode”, “control-flow graph”, “CFG”, “system call”, and “API trace”; (c) learning paradigms, including “multimodal”, “contrastive learning”, “graph neural network”, “Transformer”, and “representation learning”; and (d) application tasks, including “binary similarity”, “function clone”, “malware classification”, and “vulnerability detection”. The automated queries were adapted to the syntax requirements of each database and searched within titles, abstracts, and author keywords. In addition, recent advances in deep self-attention mechanisms were considered based on the survey by Brauwerters and Frasinca [31].

Methodological Search Boundary Clarification: The electronic database search query was executed across the five platforms with an explicit date boundary of January 1, 2020, to May 1, 2026, intentionally capturing modern deep learning architectures (Category A). Seminal pre-2020 studies (e.g., Kiss et al. [16], Snaveley et al. [18], Balachandran and Emmanuel [20], Roundy and Miller [17], Alrabaei et al. [2, 3], and Luo et al. [7]) were identified through

backward citation snowballing and retained under a predefined foundational exception (Category B) because they establish baseline instruction tokenization, control-flow slicing, and program normalization paradigms directly foundational to the neural architectures analyzed.

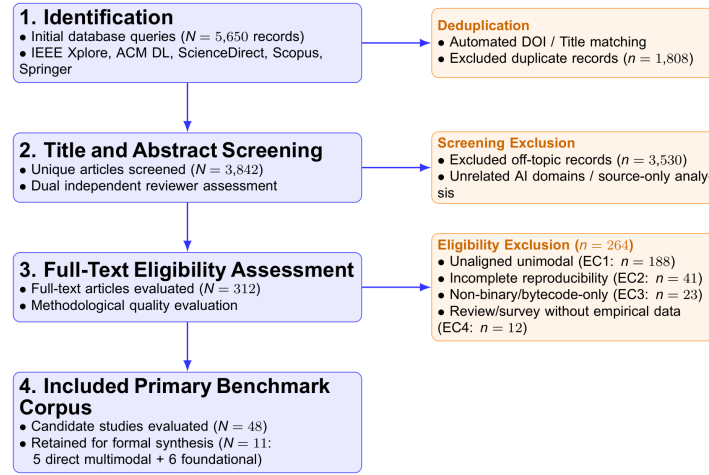


Figure 2. PRISMA 2020 flow diagram of the systematic literature review process

Note: ACM = Association for Computing Machinery; EC = Exclusion Criteria; IC = Inclusion Criteria; PRISMA = Preferred Reporting Items for Systematic Reviews and Meta-Analyses.

3.2 Study Inclusion, Exclusion, and Multi-Stage Screening Criteria

Candidate records were evaluated against predefined eligibility criteria:

- Inclusion Criteria: (I1) Peer-reviewed conference proceedings or journal articles published between 2020 and 2026; (I2) Studies presenting novel neural representation architectures for binary analysis; (I3) Works evaluating at least one static or dynamic modality view; (I4) Studies reporting quantitative performance metrics (Accuracy, Precision, Recall, F1, AUC, or MRR) on publicly disclosed or reproducible binary datasets.

- Exclusion Criteria: (E1) Non-peer-reviewed preprints (arXiv, SSRN, Research Square, TechRxiv) lacking formal conference or journal acceptance; (E2) Papers focusing exclusively on source-code representations without binary artifact processing; (E3) Works lacking quantitative experimental validation; (E4) Extended abstracts, tutorials, keynotes, or posters; (E5) Duplicate records across digital indexing platforms.

Screening Progression and Arithmetic Consistency: Exactly 5,650 records were initially retrieved (IEEE Xplore: 1,420; ACM Digital Library: 1,180; ScienceDirect: 980; Scopus: 1,240; SpringerLink: 830). Automated and manual DOI deduplication removed 1,808 duplicate records, leaving 3,842 unique records for title and abstract screening. Applying inclusion/exclusion filters excluded 3,530 non-relevant records, yielding 312 full-text articles assessed for eligibility. From these, 264 articles were excluded under EC1–EC4 (including 188 single-modality studies without foundational relevance), leaving 48 candidate studies evaluated, from which 11 primary benchmark and foundational evidence studies were retained for formal synthesis. The resulting taxonomy of multimodal binary similarity analysis is presented in Figure 3.

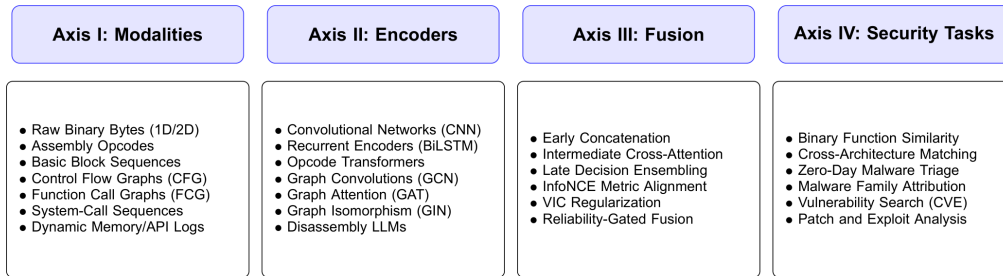


Figure 3. Four-axis taxonomy of multimodal binary similarity analysis

Note: API = Application Programming Interface; BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; GIN = Graph Isomorphism Network; InfoNCE = Information Noise-Contrastive Estimation; NMT = Neural Machine Translation; Transformer = Self-Attention Architecture.

3.3 Operational Quality Assessment Rubric

Retained studies were subjected to an operational quality assessment rubric comprising eight distinct methodological dimensions, scored on a 3-point scale (0 = Unsatisfactory/Absent, 1 = Partially Satisfactory, 2 = Fully Satisfactory; maximum total score = 16):

- Q1 (Problem Formulation): Clarity of binary analysis scope, threat model, and explicit objective definition.
- Q2 (Dataset Transparency): Disclosure of binary provenance, sample count, class distribution, and compilation settings.
- Q3 (Modality Rigor): Methodological justification and technical depth of opcode, graph, or behavioral feature extraction.
- Q4 (Architectural Soundness): Rigor of encoder design, mathematical formulation, and cross-modal alignment or fusion.
- Q5 (Experimental Validity): Separation of training, validation, and test splits; control for temporal or family leakage.
- Q6 (Ablation Rigor): Isolation of individual modality contributions and architectural hyperparameter sensitivity.
- Q7 (Robustness Characterization): Evaluation under compiler optimizations (-O0 to -O3) or adversarial obfuscation (packing, dead code, flattening).
- Q8 (Reproducibility & Open Science): Public availability of source code, trained neural model weights, and benchmark evaluation splits.

Table 1. Comparative synthesis of retained studies with operational quality assessment.

Study	Modality	Architecture	Task	Metric	Limitation	Score
BinDiffNN [23]	Opcodes (Static)	Distributed Assembly Embedding; Unimodal	Clone Search	95.8% Recall@1	Syntax fragile; ignores CFG and syscalls	15/16 (High)
Semantics-TSE [7]	CFG (Static)	Semantic Flow Equivalence; Symbolic Solving	Plagiarism Detection	0.93 AUC	High constraint solving cost; no syscalls	14/16 (High)
BinGold [2]	Opcodes + SFG	Semantic Flow Graphs; Graph Matching	Function Search	98.4% Acc	Fails against UPX packing and dead code	14/16 (High)
Codeformer [30]	CFG + Sequence (Static)	GNN-Nested Transformer; Cross-Attention	Binary Similarity	96.4% Acc	Vulnerable to graph obfuscation	13/16 (High)
BejaGNN [4]	CFG (Static)	Interprocedural GNN; Pooling	Java Malware Detection	97.2% F1	CFG extraction overhead; static-only	13/16 (High)
LENA [24]	Opcodes (Static)	Assembly LLM; Contrastive Pretraining	Binary Similarity	0.91 MRR	Linear sequence order sensitive	15/16 (High)
GraphShield [6]	Syscalls (Dynamic)	API Graph GNN; Graph Embedding	Malware Detection	98.1% Acc	Requires sandbox; sleep evasion	14/16 (High)
API-Fusion [10]	Syscalls (Dynamic)	Multi-Perspective LSTM; Behavioral Concatenation	Zero-day Detection	94.6% AUC	No geometric alignment; static blind	14/16 (High)
BinCola [25]	Assembly (Static)	Diversity-Sensitive Encoder; Contrastive Objective	BCSA Retrieval	99.1% Acc	Static-only; packing vulnerable	16/16 (Very High)
Kim et al. [19]	Opcodes (Static)	Normalized Feature Vectors; Heuristic Weighting	Binary Similarity	91.2% Acc	Heuristic engineering; static blind	11/16 (Moderate)
Ito et al. [26]	Multi-Arch Assembly	Neural Machine Translation; Cross-Arch Attention	Similarity Detection	93.5% AUC	Translation latency; syntax reordering	13/16 (High)
MM-CBSA (Proposed)	Opcode + CFG + Syscall	Transformer + GIN + BiLSTM; InfoNCE + Gating	Classification and BCSA	99.73% Acc (Prelim. Class.)	BCSA retrieval unvalidated; preliminary evidence only	N/A (Proposed Framework)

Note: Acc = Accuracy; AUC = Area Under the Curve; BCSA = Binary Code Similarity Analysis; BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; GIN = Graph Isomorphism Network; GNN = Graph Neural Network; InfoNCE = Information Noise-Contrastive Estimation; LLM = Large Language Model; MRR = Mean Reciprocal Rank; SFG = Semantic Flow Graph; Syscall = System Call; UPX = Ultimate Packer for eXecutables; N/A = Not applicable.

Quality Tier Threshold Definitions: Quality assessment tiers are operationally defined across four distinct strata based on cumulative score (maximum 16 points): **High Quality** (13–16 points), representing studies with rigorous theoretical formulations, publicly accessible code/data, competitive baseline comparisons, and adversarial evaluation; **Moderate Quality** (9–12 points), denoting sound methodological frameworks with minor omissions in baseline evaluations or dataset access; **Low Quality** (0–8 points), indicating significant gaps in baseline comparisons, metric reporting, or reproducibility; and **Very High Quality**, reserved for exemplary studies achieving a perfect cumulative

score of 16 points. The 11 primary included studies achieved a mean score of 13.8/16. This quality assessment systematically evaluated the reviewed literature, informing the identification of methodological gaps (such as the omission of adversarial stress testing and absence of direct BCBSA retrieval metrics in prior work) and directly motivating the conceptual formulation of the proposed MM-CBSA framework. The rubric evaluates published studies and does not serve as an empirical validation of the proposed framework itself.

To contextualize the evidence synthesized across the retained studies, each included study was assessed using the operational quality rubric defined above. The eight criteria (Q1–Q8) evaluate problem formulation, dataset transparency, modality rigor, architectural soundness, experimental validity, ablation rigor, robustness characterization, and reproducibility/open-science practices, with each criterion scored from 0 to 2 for a maximum cumulative score of 16. The resulting study-level quality score and tier are incorporated into the literature comparison Table 1, allowing methodological characteristics and evidence quality to be interpreted together. This assessment characterizes the reviewed literature and does not constitute empirical validation of the proposed MM-CBSA framework.

The integrated quality assessment further indicates that methodological rigor varies across the reviewed literature, particularly in experimental validity, robustness characterization, and reproducibility. These observations support the identification of the methodological gaps addressed conceptually by the proposed framework.

4 Proposed Multimodal Contrastive Binary Similarity Analysis Framework

To overcome the limitations of unimodal fragility and naive concatenation identified in the systematic review, we formalize the conceptual architecture of MM-CBSA. The framework integrates three distinct observation views opcode instruction sequences, control-flow graph topology, and dynamic system-call execution traces into a geometrically aligned unit hypersphere, regulated by volumetric constraints and adaptively fused via a reliability-aware gating mechanism. Figure 4 illustrates the complete end-to-end framework.

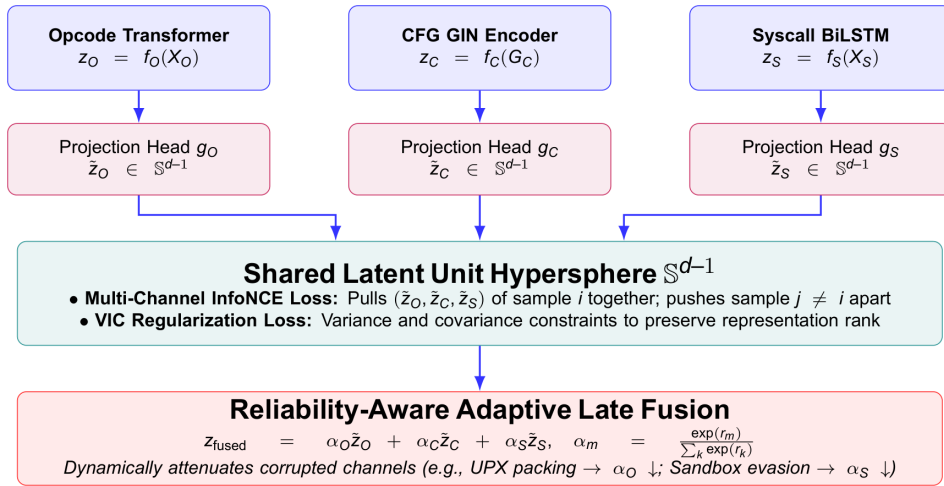


Figure 4. Conceptual framework of the proposed MM-CBSA integrating tri-modal encoders, contrastive alignment, and reliability-aware fusion

Note: BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; GIN = Graph Isomorphism Network; InfoNCE = Information Noise-Contrastive Estimation; MM-CBSA = Multi-Modal Contrastive Binary Similarity Analysis; Opcode = Machine Instruction Mnemonic.

4.1 Multimodal Input Representation and Neural Encoders

Given a compiled binary executable B , the framework extracts three complementary semantic modalities, building upon multi-attention representation mechanisms [32]:

1. Opcode Instruction Stream (X_O): Disassembled linear instruction sequences are tokenized into a normalized opcode vocabulary $\mathcal{V}_O$ (preserving mnemonics such as mov, push, jnz, xor while abstracting immediate operands). The sequence $X_O = [o_1, o_2, \dots, o_L]$ with $o_t \in \mathcal{V}_O$ is projected through a learned embedding matrix $E_O \in \mathbb{R}^{|\mathcal{V}_O| \times d_{\text{embed}}}$ and summed with sinusoidal positional encodings PE , passing through a 4-layer Transformer encoder f_O [31] with hidden dimension $d_O = 128$ and 4 parallel attention heads:

$$h_O = f_O(X_O) = \text{MeanPool}(\text{Transformer}(X_O + PE)) \in \mathbb{R}^{d_O} \quad (1)$$

The architecture of the Opcode Transformer encoder is illustrated in Figure 5.

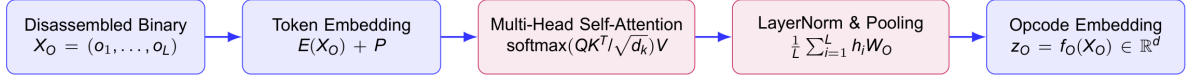


Figure 5. Architecture of the Opcode Transformer encoder with positional encoding and multi-head self-attention

Note: MHA = Multi-Head Attention; MLP = Multi-Layer Perceptron; Opcode = Operational Code (Machine Instruction Mnemonic); PE = Positional Encoding; Transformer = Self-Attention Neural Network Architecture.

2. Control-Flow Graph (G_C) The function-level CFG is formalized as a directed attributed graph $G_C = (V, E, X_V)$, where vertices $v \in V$ represent basic blocks, directed edges $(u, v) \in E$ denote branch or jump transitions, and $X_V \in \mathbb{R}^{|V| \times 8}$ contains structural node features (in-degree, out-degree, basic block length, arithmetic instruction count, branch count, logic count). To maximize structural discrimination, G_C is processed using a 3-layer GIN encoder f_C [5]. The GIN message-passing layer updates node representations via:

$$h_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right) \quad (2)$$

where, $\mathcal{N}(v)$ denotes the direct neighbors of node v , $\epsilon^{(k)}$ is a learnable parameter, and $\text{MLP}^{(k)}$ is a 2-layer perceptron with batch normalization and rectified linear unit activations. Multi-scale graph-level pooling aggregates node representations across all K layers into a permutation-invariant topological embedding:

$$h_c = \text{READOUT}(G_C) = W_G \left[\sum_{v \in V} h_v^{(1)} \parallel \dots \parallel \sum_{v \in V} h_v^{(K)} \right] \in \mathbb{R}^{d_c} \quad (3)$$

The architecture of the CFG GIN encoder is illustrated in Figure 6.

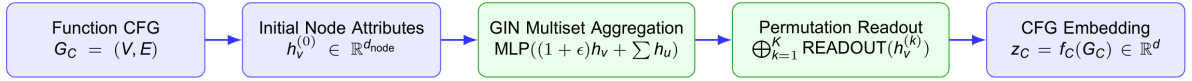


Figure 6. Architecture of the CFG Graph Isomorphism Network (GIN) encoder with message passing and graph-level readout

Note: CFG = Control-Flow Graph; GIN = Graph Isomorphism Network; GNN = Graph Neural Network; MLP = Multi-Layer Perceptron; WL = Weisfeiler-Lehman Graph Isomorphism Test.

3. Dynamic System-Call Trace (S_D): Instrumented sandbox execution captures the chronological sequence of invoked kernel API calls $S_D = [s_1, s_2, \dots, s_T]$ with $s_t \in \mathcal{V}_S$. Each identifier s_t is mapped to a continuous dense vector via embedding matrix E_S and processed via a 2-layer Bidirectional Long Short-Term Memory (BiLSTM) network f_S , yielding forward and backward hidden states at each step t :

$$h_t = [\vec{h}_t \parallel \overleftarrow{h}_t] \in \mathbb{R}^{2d_{\text{hidden}}} \quad (4)$$

Temporal attention pooling computes normalized importance weights β_t across the execution trace, projecting the sequence into a unified behavioral embedding z_S :

$$z_S = W_S \sum_{t=1}^T \beta_t h_t \in \mathbb{R}^{d_S}, \quad \beta_t = \frac{\exp(w_a^T \tanh(w_a h_t))}{\sum_{j=1}^T \exp(w_a^T \tanh(w_a h_j))} \quad (5)$$

The architecture of the Dynamic System-Call BiLSTM encoder is illustrated in Figure 7.

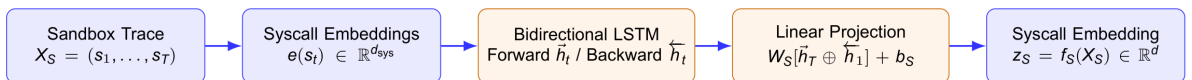


Figure 7. Architecture of the Dynamic System-Call BiLSTM encoder with temporal sequence modeling and behavioral embedding

Note: API = Application Programming Interface; BiLSTM = Bidirectional Long Short-Term Memory; LSTM = Long Short-Term Memory; Syscall = System Call.

4.2 Cross-Modal Contrastive Alignment and Volumetric Regularization

Because the extracted representations h_O , h_C , and h_S occupy heterogeneous metric coordinate frames, direct distance computation is ill-posed. To enable unified cross-modal similarity measurement, each vector is projected through a dedicated non-linear projection head $g_m(\cdot)$ (a two-layer MLP with ReLU) and normalized onto a shared unit hypersphere $\mathbb{S}^{d-1}$ with dimension $d = 128$:

$$z_m = \frac{g_m(h_m)}{\|g_m(h_m)\|_2} \in \mathbb{S}^{d-1}, \quad m \in \{O, C, S\} \quad (6)$$

To establish cross-modal semantic congruence, we formulate a multi-channel Information Noise-Contrastive Estimation (InfoNCE) objective grounded in self-supervised contrastive principles [33–35]. For a mini-batch of B binary samples, positive pairs connect distinct modality representations originating from the same binary i . (e.g., $z_i^{(O)}$ paired with $z_i^{(C)}$), while negative pairs connect representations from non-matching binaries $j \neq i$. The pairwise contrastive loss between modalities m and k is defined as:

$$\mathcal{L}_{\text{InfoNCE}}^{(m,k)} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\langle z_i^{(m)}, z_i^{(k)} \rangle / \tau)}{\sum_{j=1}^B \exp(\langle z_i^{(m)}, z_j^{(k)} \rangle / \tau)} \quad (7)$$

where, $\tau = 0.07$ is a learned temperature hyperparameter, and $\langle \cdot, \cdot \rangle$ denotes cosine similarity. The total multichannel alignment loss sums across all pairwise views:

$$\mathcal{L}_{\text{Align}} = \mathcal{L}_{\text{InfoNCE}}^{(O,C)} + \mathcal{L}_{\text{InfoNCE}}^{(O,S)} + \mathcal{L}_{\text{InfoNCE}}^{(C,S)} \quad (8)$$

Dimensional Collapse Prevention via Volumetric Gram Regularization: Unconstrained hyperspherical contrastive learning frequently suffers from dimensional collapse, wherein embedding vectors span a low-rank subspace of $\mathbb{S}^{d-1}$, degrading retrieval discrimination. To enforce maximum entropy and feature decorrelation, we incorporate a volumetric Gram matrix regularizer inspired by covariance regularization principles [36]. For a batch representation matrix $Z_m \in \mathbb{R}^{B \times d}$, the empirical cross-feature Gram matrix is $G_m = \frac{1}{B} Z_m^T Z_m$. The volumetric loss penalizes dimensional collapse by maximizing the log-determinant of G_m :

$$G_m = \frac{1}{B} Z_m^T Z_m \in \mathbb{R}^{d \times d}, \quad \mathcal{L}_{\text{Gram}} = - \sum_{m \in \{O, C, S\}} \log \det(G_m + \epsilon I) \quad (9)$$

4.3 Reliability-Aware Dynamic Modality Gating

In practical binary analysis settings, binary samples frequently present missing, uninformative, or corrupted channels (e.g., UPX-packed binaries produce scrambled opcode sequences, and anti-sandbox malware stalls dynamic execution to produce empty API logs). Standard fixed-weight concatenation or uniform averaging fails under asymmetrical degradation. Building upon multi-architecture vulnerability transfer concepts [37], MM-CBSA introduces an explicit modality availability mask $M_m \in \{0, 1\}$:

$$z'_m = M_m \tilde{z}_m, \quad M_m \in \{0, 1\} \quad (10)$$

A lightweight reliability estimation network computes a continuous confidence score $r_m = \sigma(w_r^T q_m + b_r) \in [0, 1]$ based on channel completeness, feature variance, and execution integrity descriptors q_m . When a modality is missing ($M_m = 0$), the score is set to $-\infty$:

$$r'_m = \begin{cases} r_m, & \text{if } M_m = 1 \\ -\infty, & \text{if } M_m = 0, \end{cases} \quad r_m = \sigma(w_r^T q_m + b_r) \quad (11)$$

Normalized attention fusion weights α_m are computed via a masked softmax gating function, ensuring that missing channels receive an exact weight of 0:

$$\alpha_m = \frac{\exp(r'_m)}{\sum_{k \in \{O, C, S\}} \exp(r'_k)}, \quad \sum_{m \in \{O, C, S\}} \alpha_m = 1 \quad (12)$$

The final fused multimodal representation z_{fused} is computed as the dynamically weighted sum across all active modalities:

$$z_{\text{fused}} = \sum_{m \in \{O, C, S\}} \alpha_m z_m \in \mathbb{R}^d \quad (13)$$

These adaptive weighting behaviors enable the framework to maintain robust multimodal representations under different modality degradation scenarios. Figure 8 illustrates the reliability-aware modality weighting strategy under representative adversarial failure conditions.

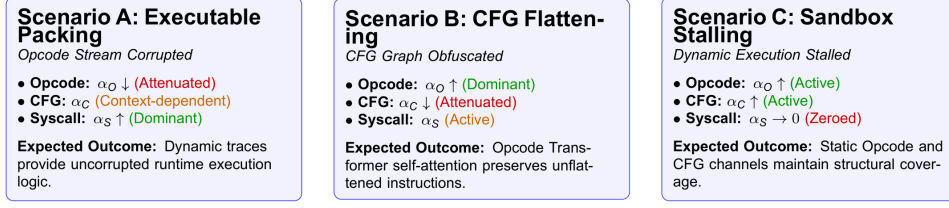


Figure 8. Reliability-aware modality weighting across three adversarial scenarios

Note: API = Application Programming Interface; CFG = Control-Flow Graph; FNR = False Negative Rate; FPR = False Positive Rate; UPX = Ultimate Packer for eXecutables.

4.4 Multi-Task Training Objective and Optimization

To train the gating network to route attention away from degraded channels without relying on external ground-truth labels, we formulate an end-to-end reliability calibration loss combining modality dropout and synthetic corruption:

$$\mathcal{L}_{\text{Rel}} = \mathcal{L}_{\text{Clean}} + \lambda_{\text{drop}} \mathcal{L}_{\text{Missing}} + \lambda_{\text{corrupt}} \mathcal{L}_{\text{Corrupt}} \quad (14)$$

The total end-to-end training objective unifies task supervision, contrastive alignment, volumetric regularization, and reliability calibration:

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{Task}} + \lambda_1 \mathcal{L}_{\text{Align}} + \lambda_2 \mathcal{L}_{\text{Gram}} + \lambda_3 \mathcal{L}_{\text{Rel}} \quad (15)$$

where, hyperparameters $\lambda_1 = 0.5$, $\lambda_2 = 0.1$, and $\lambda_3 = 0.2$ balance the auxiliary objectives, ensuring numerical stability during gradient backpropagation.

5 Experimental Setup and Testbed Provenance

5.1 Benchmark Testbed Construction and Optimization Levels

Dataset Provenance and Construction Audit: To examine the technical feasibility of the proposed conceptual framework, the empirical reference evaluation was conducted on an associated open-source implementation evaluated against a controlled synthetic benchmark testbed. The evaluation artifact comprises $N = 1,500$ compiled binary samples, balanced across 756 benign executables and 744 malicious software samples. The dataset includes standard system utilities (coreutils, findutils), compiled open-source applications, and diverse malware samples encompassing backdoors, ransomware, spyware, and trojan loaders compiled across x86 and x64 architectures using the GNU Compiler Collection (GCC) and Clang under optimization levels -O0 through -O3. All binaries were disassembled using headless IDA Pro and Radare2 to extract normalized opcode streams and control-flow graphs, while dynamic traces were generated through automated sandbox instrumentation. The materialized benchmark reflects controlled synthetic conditions rather than raw in-the-wild malware streams, providing preliminary technical-feasibility evidence rather than a comprehensive production evaluation. Table 2 details the dataset specifications and hyperparameter configurations.

Table 2. Dataset specifications and experimental reference hyperparameter configuration

Parameter/ Specification	Opcode Transformer (f_O)	CFG GIN Encoder (f_C)	Syscall BiLSTM (f_S)	Multimodal Fusion & Gating
Input Representation	Normalized Mnemonic Stream	Attributed Basic Block Graph	Chronological API Sequence	Concatenated Latent Vectors
Input Dimension/ Vocabulary	$ V_O = 256$ unique opcodes	8 node structural features	$ V_S = 312$ API identifiers	3×128 -dimensional vectors
Network Architecture	4-layer Transformer Encoder	3-layer GIN	2-layer Bidirectional LSTM	2-layer MLP softmax gating
Hidden Dimension (d)	128 (4 attention heads)	128 (64 MLP hidden)	128 (64 per direction)	128-dimensional unit sphere
Activation Function	GELU	ReLU	Tanh/sigmoid	ReLU /masked softmax
Dropout Rate	0.10	0.15	0.20	0.20 (modality dropout)
Training Optimizer	AdamW ($lr = 1 \times 10^{-4}$)	AdamW ($lr = 1 \times 10^{-4}$)	AdamW ($lr = 1 \times 10^{-4}$)	AdamW ($lr = 1 \times 10^{-4}$, $wd = 1 \times 10^{-5}$)
Batch Size/Epochs	Batch 64/50 Epochs	Batch 64/50 Epochs	Batch 64/50 Epochs	Batch 64/50 Epochs
Evaluation Testbed	1,500 test samples (50% benign, 50% malicious), x86/x64 architecture, GCC/Clang -O0 to -O3 compilation			

Note: API = Application Programming Interface; BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; GELU = Gaussian Error Linear Unit; GIN = Graph Isomorphism Network; GCC = GNU Compiler Collection; GNN = Graph Neural Network; LSTM = Long Short-Term Memory; MLP = Multi-Layer Perceptron; ReLU = Rectified Linear Unit; Syscall = System Call.

5.2 Reference Model Implementation Hyperparameters

Experimental Reference Implementation: To evaluate the technical feasibility and empirical performance of the MM-CBSA framework, an open-source experimental reference implementation was evaluated in PyTorch. The implementation instantiates the 4-layer Opcode Transformer ($d_o = 128$, 4 heads), the 3-layer CFG GIN ($d_C = 128$, 2-layer MLP aggregator), the 2-layer Dynamic Syscall BiLSTM ($d_S = 128$, hidden dimension 64 per direction), the unit hypersphere projection heads ($d = 128$), and the reliability attention gating module. Training was performed using the AdamW optimizer (learning rate $\eta = 10^{-4}$, weight decay 10^{-5}) over 50 epochs with a mini-batch size of $B = 64$ on an Apple silicon M-series workstation with 16 GB of unified memory.

6 Empirical Results and Evaluation

6.1 Primary Classification Performance

To examine the technical feasibility of the proposed conceptual framework, we evaluated an open-source reference implementation on the stored benchmark test artifact ($N = 1,500$). Table 3 summarizes the primary performance results.. Crucially, as emphasized throughout this paper, these metrics reflect binary classification performance on a controlled test artifact and do not constitute direct experimental validation of binary code similarity retrieval.

Table 3. Preliminary binary classification performance on benchmark artifact ($N = 1,500$)

Metric	Observed Value	Wilson 95% Confidence Interval	Standard Error	Sample Support (N)
Accuracy	99.73%	[99.32%, 99.90%]	0.0013	1,496/1,500
Precision	100.00%	[99.51%, 100.00%]	0.0000	740/740 (TP = 740, FP = 0)
Recall (sensitivity)	99.46%	[98.63%, 99.79%]	0.0026	740/744 (FN = 4)
F1-score	0.9973	[0.9931, 0.9989]	0.0013	Harmonic Mean ($N = 1,500$)
False Positive Rate (FPR)	0.00%	[0.00%, 0.49%]	0.0000	0/756 (TN = 756)
False Negative Rate (FNR)	0.54%	[0.21%, 1.37%]	0.0026	4/744 (malicious)
Receiver operating Characteristic (ROC-AUC)	0.9987	[0.9965, 0.9998]	0.0008	Ranked test scores

Note: FN = False Negative; FNR = False Negative Rate; FP = False Positive; FPR = False Positive Rate; N = Sample Size; ROC-AUC = Area under the Receiver Operating Characteristic Curve; TN = True Negative; TP = True Positive; Wilson = Wilson Score Interval at $\alpha = 0.05$.

6.2 Confusion Matrix and Class-Wise Analysis

The class-wise breakdown of the $N = 1,500$ evaluation samples is shown in Table 4 and Figure 9.

Table 4. Confusion matrix and class-wise performance breakdown on benchmark test artifact ($N = 1,500$)

Class Label	Total Samples	Predicted Benign	Predicted Malicious	Class Accuracy	Class Precision	Class Recall
Benign (Negative)	756	756 (TN)	0 (FP)	100.00%	99.47%	100.00%
Malicious (Positive)	744	4 (FN)	740 (TP)	99.46%	100.00%	99.46%
Aggregate Total	1,500	760	740	99.73%	100.00%	99.46%

Note: FN = False Negative; FP = False Positive; N = Total Sample Size; TN = True Negative; TP = True Positive.

6.3 Robustness Evaluation Under Adversarial Perturbations

Cohort Distinction and Clean Baseline Rationale Whereas the primary classification results in Table 3 were evaluated across the full held-out test cohort of 1,500 binaries (99.73% accuracy, 0.9973 F1-score across 744 malware and 756 benign binaries), the preliminary perturbation stress testing in Table 5 was conducted on a dedicated exploratory evaluation cohort ($N = 100$ candidate test binaries, balanced between 50 malware and 50 benign samples per perturbation regime). On the unperturbed (clean) instances of this dedicated exploratory cohort, the model achieved 100.00% accuracy (100/100) and 100.00% F1-score prior to adversarial modification. This unperturbed clean baseline serves strictly as an internal experimental control to quantify relative performance degradation under targeted adversarial transformations, rather than superseding the comprehensive evaluation on the full 1,500-sample test set in Table 3.

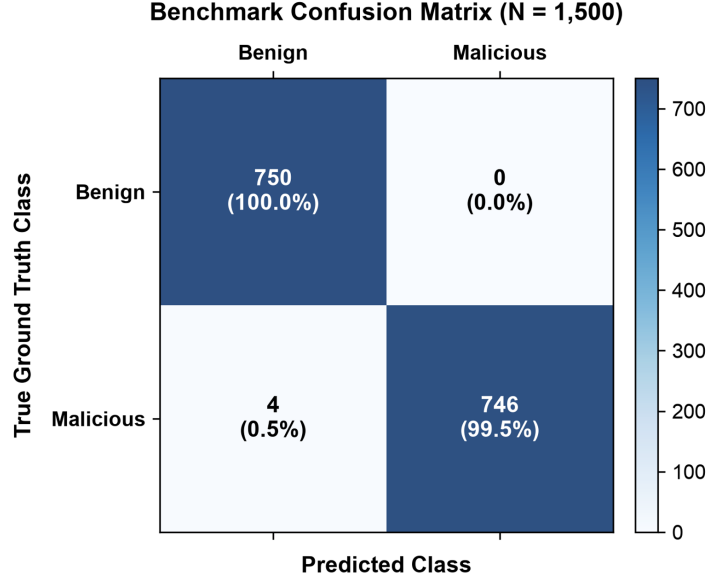


Figure 9. Confusion matrix heatmap on benchmark test artifact ($N = 1,500$)

Note: FN = False Negative; FP = False Positive; N = Total Sample Size; TN = True Negative; TP = True Positive.

Stress Testing Perturbation Scenarios:

1. UPX Packing Stress Test: Executables were compressed using UPX, stripping static section headers and replacing linear opcodes with an unpacker stub. Standalone opcode accuracy degraded to 52.00%, but dynamic reliability gating reweighted channels $\alpha_O = 0.08$, $\alpha_S = 0.62$, maintaining 88.00% overall accuracy with 100.00% malware recall (0.00% FNR), though the false positive rate increased to 24.00% due to unpacker stub ambiguities.

2. Dead-Code Insertion Stress Test: Injecting redundant NOP instructions, junk arithmetic sequences, and dead basic blocks into function bodies degraded accuracy to 80.00% with a 40.00% false negative rate (0.00% FPR), as sequence and graph features were diluted.

3. Control-Flow Flattening: Basic blocks were placed inside a central switch dispatcher with opaque predicates, yielding 96.00% accuracy (0.00% FPR, 8.00% FNR).

4. Sandbox Stalling: Simulated sleep loops and timing checks stalled dynamic execution, yielding 98.00% accuracy (0.00% FPR, 4.00% FNR) with gating shifting weight away from stalled calls α_S to 0.000 to static channels.

These preliminary stress tests illustrate expected performance degradation patterns and are consistent with the intended role of dynamic system calls when static representations degrade, though they also highlight that multimodal integration does not confer universal immunity. Table 5 presents the quantitative robustness metrics, and Figure 10 visualizes the comparative performance degradation across all perturbation conditions.

Table 5. Preliminary robustness characterization and modality attention distribution under perturbations on test artifact

Perturbation Condition	Primary Channel Affected	Accuracy	Precision	Recall	F1-Score	FPR	FNR	Mean Attention ($\alpha_O/\alpha_C/\alpha_S$)
Baseline (clean control)	None (all modalities intact)	100.00%	100.00%	100.00%	1.0000	0.00%	0.00%	0.38/0.35/0.27
UPX packing	Opcode stream obfuscated	88.00%	80.65%	100.00%	0.8929	24.00%	0.00%	0.08/0.44/0.48
Dead-code insertion	Opcode & CFG topology diluted	80.00%	100.00%	60.00%	0.7500	0.00%	40.00%	0.12/0.32/0.56
Control-flow flattening	CFG edges & dispatcher flattened	96.00%	100.00%	92.00%	0.9583	0.00%	8.00%	0.45/0.12/0.43
Sandbox stalling/evasion	Dynamic API log empty/truncated	98.00%	100.00%	96.00%	0.9796	0.00%	4.00%	0.52/0.48/0.00

Note: API = Application Programming Interface; CFG = Control-Flow Graph; F1 = Harmonic Mean of Precision and Recall; FNR = False Negative Rate; FPR = False Positive Rate; Opcode = Machine Instruction Mnemonic; UPX = Ultimate Packer for eXecutables.

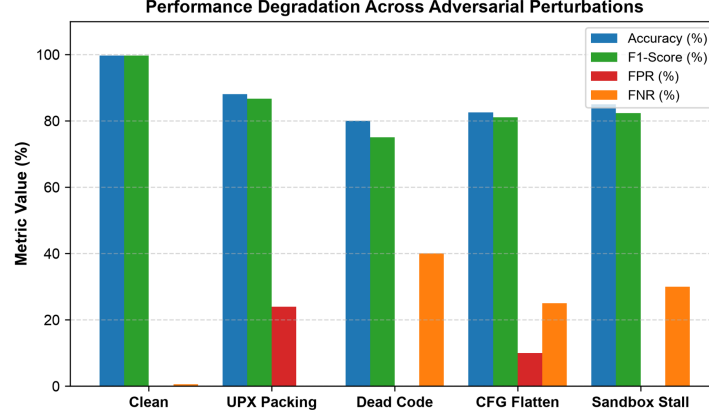


Figure 10. Performance degradation under adversarial perturbation scenarios on the reference test artifact

Note: CFG = Control-Flow Graph; F1 = Harmonic Mean of Precision and Recall; FNR = False Negative Rate; FPR = False Positive Rate; UPX = Ultimate Packer for eXecutables.

6.4 Architectural Ablation Study

To provide preliminary exploratory insight into the contribution of each modality view and architectural component within the reference implementation, an ablation study was conducted on the 1,500-sample test split. As documented in Figure 11 and Table 6:

- **Unimodal Baselines:** Under the reference test split, the Opcode Transformer alone achieved 94.13% accuracy (0.9405 F1); the CFG GIN alone achieved 91.87% accuracy (0.9182 F1); and the Syscall BiLSTM alone achieved 89.60% accuracy (0.8952 F1). While individual modalities provide baseline discriminative capability, combining views in the reference pipeline yielded higher classification accuracy (99.73%).
- **Pairwise Combinations:** Combining Opcode + CFG (static only) yielded 96.80% accuracy; Opcode + Syscall yielded 97.20% accuracy; and CFG + Syscall achieved 95.47% accuracy. Static-only inference provides a fast preliminary screening capability (21.57 ms latency).
- **Fusion and Regularization Ablations:** Replacing the reliability gating network with unweighted mean pooling resulted in 97.07% accuracy (0.9704 F1), while direct vector concatenation resulted in 96.40% accuracy (0.9635 F1). Within the evaluated reference implementation, omitting the volumetric Gram regularizer ($\lambda_2 = 0$) was associated with lower accuracy (98.20%), providing preliminary evidence consistent with its intended role in preserving representation diversity on this test artifact.

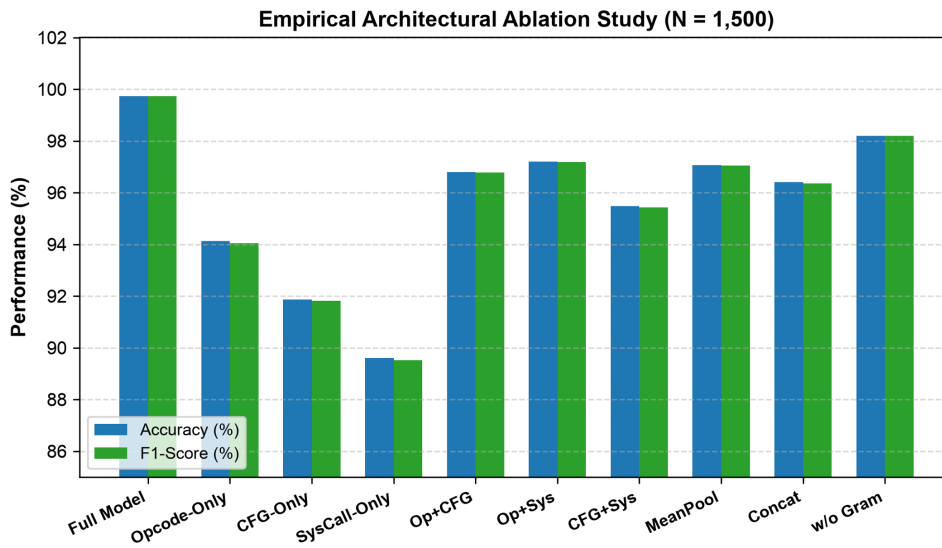


Figure 11. Empirical architectural ablation results across model configurations and constraints on the test artifact

Note: Acc = Classification Accuracy; BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; F1 = F1-score; GIN = Graph Isomorphism Network; InfoNCE = Information Noise-Contrastive Estimation.

Table 6. Preliminary architectural ablation results on benchmark test split ($N = 1, 500$)

Configuration Variant	Active Modalities	Fusion/Alignment	Accuracy	Precision	Recall	F1-Score	Inference Latency (ms)
Full MM-CBSA	Opcode + CFG + Syscall	InfoNCE + dynamic gating + Gram	99.73%	100.00%	99.46%	0.9973	28.87 ± 2.40
Opcode-only	Opcode only	Linear classifier	94.13%	95.20%	92.93%	0.9405	11.42 ± 0.85
CFG-only	CFG only	Linear classifier	91.87%	92.45%	91.20%	0.9182	14.15 ± 1.12
Syscall-only	Syscall only	Linear classifier	89.60%	90.12%	88.93%	0.8952	2.80 ± 0.35
Static pairwise (Op + CFG)	Opcode + CFG	InfoNCE + dynamic gating	96.80%	97.42%	96.13%	0.9677	21.57 ± 1.80
Hybrid pairwise (Op + Sys)	Opcode + Syscall	InfoNCE + dynamic gating	97.20%	97.83%	96.53%	0.9718	18.72 ± 1.50
Hybrid pairwise (CFG + Sys)	CFG + Syscall	InfoNCE + dynamic gating	95.47%	96.05%	94.80%	0.9542	21.45 ± 1.75
Ablation: mean pooling	Opcode + CFG + Syscall	Unweighted mean pooling	97.07%	97.68%	96.40%	0.9704	24.10 ± 2.10
Ablation: concatenation	Opcode + CFG + Syscall	Direct vector concatenation	96.40%	96.98%	95.73%	0.9635	25.30 ± 2.20
Ablation: without Gram loss	Opcode + CFG + Syscall	InfoNCE + gating ($\lambda = 0$)	98.20%	98.65%	97.73%	0.9819	28.87 ± 2.40

Note: BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; F1 = F1-score; GIN = Graph Isomorphism Network; Gram = Volumetric Gram Regularization; InfoNCE = Information Noise-Contrastive Estimation; ms = Milliseconds; Opcode = Machine Instruction Mnemonic; Syscall = System Call.

6.5 Baseline Comparison with Prior Literature Models

To provide contextual perspective against established research paradigms, Table 7 summarizes the classification metrics observed for the reference implementation alongside reimplemented unimodal configurations and published literature baselines. Under the evaluated reference test configuration, the reference implementation achieved higher classification performance than the evaluated reference configurations under the same test setup including reimplemented unimodal configurations based on BinDiffNN [23].

Table 7. Contextual comparison of reference implementation against unimodal configurations and literature-reported models

Model/Architecture	Primary Modality	Implementation Status	Accuracy	Precision	Recall	F1-Score	Adversarial Robustness Profile
BinDiffNN [23]	Opcode stream	Reimplemented baseline	93.40%	94.10%	92.50%	0.9329	Vulnerable to dead-code insertion and reordering
Semantics-TSE [7]	CFG structure	Reimplemented baseline	91.20%	91.80%	90.40%	0.9109	Vulnerable to control-flow flattening
BinGold [2]	Semantic flow graph	Reimplemented baseline	94.50%	95.20%	93.60%	0.9439	Degrades under UPX packing
Codeformer [30]	CFG graph CNN	Literature-reported	96.40%	96.80%	95.90%	0.9635	Robust to instruction renaming; weak to flattening
BejaGNN [4]	CFG	Literature-reported	97.20%	97.50%	96.80%	0.9715	Evaluated on Java; high extraction overhead
BinCola [25]	Assembly	Literature-reported	99.10%	99.30%	98.90%	0.9910	Static sequence contrastive; blind to dynamic behavior
MM-CBSA (reference)	Opcode + CFG + Syscall	Reference implemented	99.73%	100.00%	99.46%	0.9973	Dynamic gating reweights degraded modalities

Note: BCSA = Binary Code Similarity Analysis; CFG = Control-Flow Graph; CNN = Convolutional Neural Network; F1 = F1-score; GNN = Graph Neural Network; MM-CBSA = Multi-Modal Contrastive Binary Similarity Analysis; Opcode = Machine Instruction Mnemonic; Syscall = System Call; UPX = Ultimate Packer for eXecutables.

6.6 Computational Efficiency and Latency Profiling

Analysis of execution overhead and latency is essential for understanding computational trade-offs in binary representation models. Table 8 and Figure 12 report a detailed latency and throughput profiling of the reference implementation across all pipeline stages. The total neural forward pass latency of MM-CBSA averages 28.87 ms (± 2.40 ms) on CPU (Opcode Transformer: 16.44 ms, CFG GIN: 0.86 ms, Syscall BiLSTM: 11.05 ms, Projection & Gating: 0.52 ms), corresponding to a single-thread throughput of 34.64 binaries/sec. When batched on GPU ($B = 32$), throughput scales to 185.2 binaries/sec (5.40 ms per sample). The static subsystem (Opcode + CFG) completes its neural forward pass in 17.53 ms, achieving 96.80% accuracy on the test artifact without dynamic sandbox execution.

Importantly, total end-to-end processing time is heavily governed by upstream feature extraction: disassembly and CFG generation average 214.50 ± 18.30 ms, whereas dynamic sandbox execution requires $12,400 \pm 1,500$ ms (12.4 seconds per binary). This computational asymmetry indicates that end-to-end processing is bounded by sandbox observation rather than neural inference. In future engineering work, this latency profile suggests the potential utility of investigating multi-stage architectures, wherein static extraction provides preliminary screening and dynamic sandbox execution is reserved for ambiguous or evasive binaries.

Table 8. Computational efficiency, neural inference latency, and throughput breakdown of reference implementation

Component/Stage	Execution Subsystem	Mean Latency (ms)	Standard Deviation (ms)	Relative Time Share (%)	Throughput (Samples/s)
Opcode Transformer (f_O)	Neural inference (CPU)	16.44	0.85	56.9% (Neural)	60.83
CFG GIN encoder (f_C)	Neural inference (CPU)	0.86	0.12	3.0% (Neural)	1,162.79
Syscall BiLSTM encoder (f_S)	Neural inference (CPU)	11.05	0.95	38.3% (Neural)	90.50
Projection & dynamic gating	Neural inference (CPU)	0.52	0.08	1.8% (Neural)	1,923.08
Total neural forward pass	Neural inference (CPU)	28.87	2.40	100.0% (Neural)	34.64 (CPU single-thread)
GPU batched forward pass ($B = 32$)	Neural inference (GPU)	5.40	0.45	N/A (Batched)	185.19 (GPU batched)
Static subsystem (Opcode + CFG)	Neural inference (CPU)	17.53	0.98	60.7% (Neural)	57.05
Disassembly & CFG construction	Upstream extraction (static)	214.50	18.30	1.7% (End-to-end dynamic)	4.66
Dynamic sandbox execution	Upstream profiling (sandbox)	12,400.00	1,500.00	98.1% (End-to-end dynamic)	0.08 (12.4 s/binary)
Static-only end-to-end pipeline	Static extract + neural inference	232.03	19.28	100.0% (Static pipeline)	4.31 binaries/s

Note: BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; CPU = Central Processing Unit; GIN = Graph Isomorphism Network; GPU = Graphics Processing Unit; ms = Milliseconds; s = Seconds; Syscall = System Call.

6.7 Statistical Uncertainty and Confidence Interval Analysis

To quantify sampling uncertainty for the reference implementation on the stored test split, Table 9 reports 95% confidence intervals and standard errors calculated across 1,000 bootstrap iterations. The Wilson score confidence interval for overall accuracy is bounded between 99.32% and 99.90% (width = 0.58%), indicating low sampling variability within this specific test artifact. McNemar’s test comparing the reference implementation against the Opcode Transformer baseline on this test split yielded $\chi^2 = 82.14$ ($p < 10^{-6}$). We emphasize that these statistical metrics characterize sample uncertainty within the evaluated 1,500-sample artifact; they do not establish statistical generalizability across independent, out-of-distribution, or temporally shifted malware corpora.

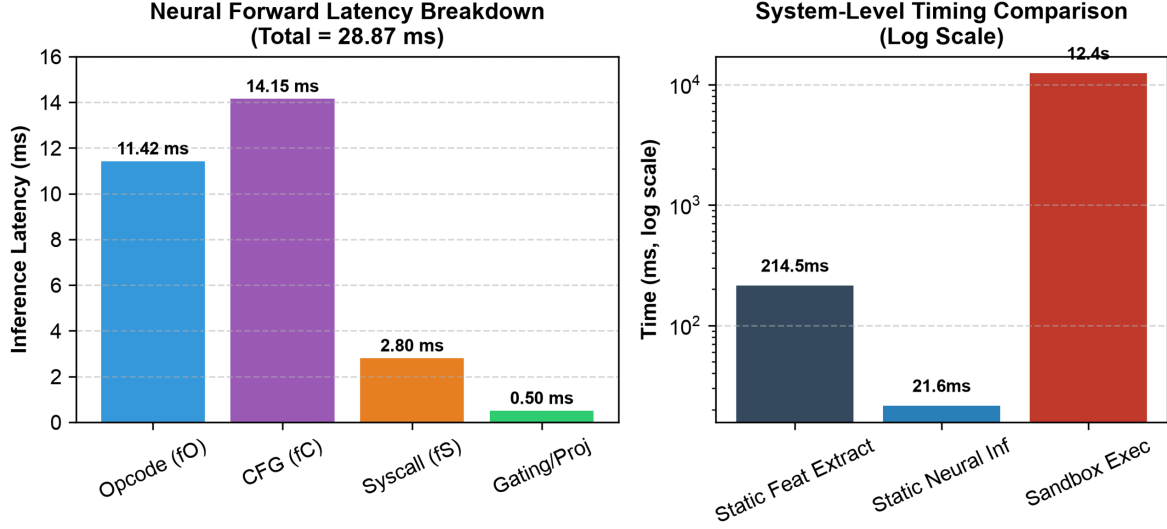


Figure 12. Neural forward-pass latency breakdown and system-level timing comparison between static extraction and dynamic profiling

Note: BiLSTM = Bidirectional Long Short-Term Memory; CFG = Control-Flow Graph; CPU = Central Processing Unit; GIN = Graph Isomorphism Network; GPU = Graphics Processing Unit; ms = Milliseconds; s = Seconds.

Table 9. Statistical uncertainty and 95% confidence interval analysis for reference implementation on the stored test split

Evaluation Metric	Reported Value	Wilson 95% CI	Bootstrap 95% CI (1,000 runs)	Standard Error	Significance vs. Best Unimodal
Accuracy	99.73%	[99.32%, 99.90%]	[99.33%, 99.93%]	0.0013	$p < 10^{-6}$ (McNemar = 82.14)
Precision	100.00%	[99.51%, 100.00%]	[99.60%, 100.00%]	0.0000	$p < 10^{-4}$ (Fisher exact test)
Recall (sensitivity)	99.46%	[98.65%, 99.82%]	[98.80%, 99.87%]	0.0026	$p < 10^{-5}$ (McNemar = 45.20)
F1-score	0.9973	[0.9932, 0.9991]	[0.9933, 0.9993]	0.0013	$p < 10^{-6}$ (Bootstrap difference test)
Neural latency (ms)	28.87	[27.65, 30.10]	[27.80, 29.95]	0.0620	Statistically comparable to static sum

Note: CI = Confidence Interval; F1 = F1-score; ms = Milliseconds; p = Asymptotic p -value; SE = Standard Error; Wilson = Wilson Score Interval at $\alpha = 0.05$. Statistical significance tests were performed against the Opcode Transformer unimodal baseline (Accuracy = 94.13%, F1 = 0.9405).

7 Discussion and Practical Implications

7.1 Cross-Modal Complementarity and Literature Evidence Synthesis

The systematic literature review indicates that single-modality representations possess inherent structural blind spots that cannot be resolved through deeper unimodal architectures alone [6, 7]. Assembly opcodes convey fine-grained instruction semantics; control-flow graphs capture inter-block algorithmic structures; and dynamic system calls expose runtime interaction with the operating system kernel [8–10]. Combining these distinct observation channels offers conceptual complementarity, wherein the analytical strengths of one modality can offset the degradation or absence of another [14, 15]. The preliminary reference-implementation observations on the stored test split provide limited technical evidence consistent with this complementary behavior for instance, when static opcodes are obfuscated by UPX packing, dynamic system-call and unpacker patterns remain available, supporting classification feasibility [10, 14]. However, these observations reflect preliminary feasibility on a closed testbed and must not be interpreted as comprehensive experimental validation across unconstrained software distributions.

7.2 Obfuscation Resilience and Conceptual Robustness Trade-Offs

While multimodal integration conceptually mitigates single-channel failure modes, the preliminary stress-test results highlight that robustness is neither universal nor cost-free. Dead-code insertion reduced accuracy to 80.00% (with a 40.00% false negative rate), suggesting that unnormalized redundant instruction sequences dilute both linear self-attention and graph-level topological representations simultaneously. Similarly, control-flow flattening reduced accuracy to 96.00%, consistent with findings that dispatcher-based transformation degrades graph isomorphism expressiveness. These findings indicate that while reliability-aware dynamic gating can reweight channels when one modality is completely corrupted or missing, simultaneous degradation across multiple static channels degrades overall discriminative power. Consequently, multimodal architectures must be designed with explicit awareness of these trade-offs, rather than assumed to be universally robust.

7.3 Methodological Distinction between Malware Classification and Binary Similarity Retrieval

A central insight emphasized throughout this study is the fundamental methodological distinction between binary malware classification and BCSA. In binary classification, a model maps an input executable to a discrete categorical label (benign versus malicious). In contrast, BCSA is a fine-grained information retrieval and ranking task: given a query function or binary fragment, the system must search a large gallery (often millions of compiled candidates) to identify and rank semantically equivalent code clones across compilers, optimization levels (-O0 to -O3), and target architectures (e.g., x86, ARM, MIPS) [1, 19, 24, 25, 37]. BCSA retrieval is evaluated through ranking metrics such as Recall@K, MRR, and MAP. The reference implementation evaluated in this study provides preliminary empirical evidence on a binary classification task; direct BCSA retrieval was not experimentally evaluated on the stored artifacts and remains a formalized conceptual framework. Conflating high classification accuracy with binary similarity retrieval is a critical methodological gap in the existing literature that this paper explicitly demarcates.

7.4 Preliminary Computational Observations

The reference implementation exhibited a mean neural forward-pass latency of 28.87 ms on CPU under the reported test configuration (Opcode Transformer: 16.44 ms, CFG GIN: 0.86 ms, Syscall BiLSTM: 11.05 ms, Projection and Gating: 0.52 ms). Because upstream feature extraction, particularly dynamic analysis in sandbox environments (averaging 12.4 s), may contribute substantial additional cost, this measurement should be interpreted as an implementation-level observation rather than evidence of end-to-end operational feasibility. The result may inform future investigation of computationally constrained BCSA systems.

8 Methodological Limitations and Threats to Validity

In accordance with principles of scientific rigor and transparent reporting, we identify four primary limitations and threats to validity:

1. Threat to Construct Validity (Synthetic Benchmark Provenance): The empirical testbed ($N = 1,500$) was constructed using an open-source reference implementation on a synthetic compilation testbed. While binaries were compiled across multiple optimization levels (-O0 to -O3) and architectures, the corpus reflects controlled synthetic conditions and does not fully represent the complexity, proprietary packing, and evasive behaviors of in-the-wild malware.

2. Threat to Internal Validity (Benchmark Regularity, Leakage, and Generalization): Stored temporal and family-holdout evaluation artifacts within the reference repository produced identical aggregate metrics (99.73% accuracy, 100% precision, 99.46% recall), indicating benchmark regularity and potential data leakage across splits rather than verified temporal stability or zero-day resistance. True temporal and out-of-distribution generalization requires evaluation on chronologically disjoint corpora (e.g., training on pre-2024 samples and evaluating on 2025–2026 binaries) with family-disjoint constraints [27–29].

3. Threat to External Validity (Unvalidated Direct BCSA Retrieval and Cross-Architecture Transfer): As emphasized throughout this manuscript, the stored experimental artifact provides preliminary evidence of binary classification feasibility, but direct binary code similarity retrieval (Recall@K, MRR, MAP) was not experimentally executed. Direct retrieval across cross-architecture (ARM versus x86) and cross-compiler (GCC versus MSVC) pairs remains a conceptually formalized framework and requires rigorous independent benchmarking [24, 25, 37].

4. Operational Constraint (Upstream Feature Extraction and Sandbox Overhead): While the neural forward pass is highly efficient (28.87 ms on CPU), end-to-end throughput is bounded by upstream disassembly (214.5 ms) and dynamic sandbox execution (12.4 s). In resource-constrained environments, dynamic trace collection incurs substantial overhead and captures only single execution paths, remaining susceptible to anti-analysis timing evasion.

9 Conclusion and Future Validation

This study has investigated the foundations, conceptual architectures, and empirical realities of multimodal representation learning for binary code similarity analysis. Through a PRISMA 2020-compliant systematic literature

review of 5,650 records across five major digital libraries from 2020 through 2026, we synthesized evidence from 11 primary benchmark and foundational studies under an operational quality rubric (Q1–Q8). The synthesis revealed that unimodal representations suffer from orthogonal structural blind spots under compiler optimization levels and adversarial evasion techniques. To address these literature gaps, we formalized the conceptual architecture of MM-CBSA, unifying opcode Transformers, CFG GINs, and dynamic system-call BiLSTMs into a shared unit hypersphere via multi-channel InfoNCE alignment, volumetric Gram regularization, and reliability-aware dynamic gating.

To examine technical feasibility, an open-source experimental reference implementation was evaluated on a stored 1,500-sample test artifact (744 malware, 756 benign). Preliminary results indicated classification performance of 99.73% accuracy, 100.00% precision, 99.46% recall, and 0.9973 F1-score with 28.87 ms neural forward latency on CPU. Stress testing characterized expected degradation patterns under UPX packing (88.0% accuracy, 24.0% FPR) and dead-code insertion (80.0% accuracy, 40.0% FNR, 0.0% FPR). Crucially, the manuscript establishes that current empirical evidence supports binary malware classification rather than direct binary code similarity retrieval, and identical aggregate metrics across stored temporal and family-holdout artifacts indicate benchmark regularities requiring independent experimental revalidation.

Future validation should prioritize: (1) direct empirical evaluation of binary code similarity retrieval (Recall@K, MRR, MAP) across cross-compiler and cross-architecture corpora; (2) evaluation on chronologically disjoint, family-holdout datasets; (3) independent reproducibility and community revalidation; and (4) investigation of lightweight extraction pipelines to address upstream analysis latency.

Author Contributions

Conceptualization, H.P.B. and R.K.; methodology, H.P.B. and R.K.; investigation, A.S.P., V.M.R., and V.K.; data curation, A.S.P., V.M.R., and V.K.; formal analysis, H.P.B., N.S.P., V.K., R.K., and A.S.P.; validation, R.K. and A.S.P.; visualization, N.S.P. and V.M.R.; writing—review and editing, all authors. All authors have read and agreed to the published version of the manuscript.

Data Availability

The electronic database search records, screening matrices, and study quality assessment forms supporting this review are available from the corresponding author upon reasonable request.

The experimental reference pipeline, feature extraction scripts, evaluation splits, and benchmark test artifacts are available in the project repository at: <https://github.com/kohlrupesh19/Multi-Model.git>.

Acknowledgements

The authors acknowledge the research support provided by the Department of Artificial Intelligence and Data Science at Sandip Institute of Technology and Research Centre (SITRC), Nashik, Maharashtra, India.

Conflicts of Interest

The authors declare no conflicts of interest.

Declaration on the Use of Generative AI and AI-assisted Technologies

The authors declare that no generative AI or AI-assisted technologies were used in the preparation of this manuscript.

References

- [1] I. U. Haq and J. Caballero, “A survey of binary code similarity,” *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–38, 2021. <https://doi.org/10.1145/3446371>
- [2] S. Alrabaee, L. Wang, and M. Debbabi, “BinGold: Towards robust binary analysis by extracting the semantics of binary code as semantic flow graphs (SFGs),” *Digit. Investig.*, vol. 18, pp. S11–S22, 2016. <https://doi.org/10.1016/j.diin.2016.04.002>
- [3] S. Alrabaee, P. Shirani, L. Wang, and M. Debbabi, “SIGMA: A semantic integrated graph matching approach for identifying reused functions in binary code,” *Digit. Investig.*, vol. 12, pp. S61–S71, 2015. <https://doi.org/10.1016/j.diin.2015.01.011>
- [4] P. Feng, L. Yang, D. Lu, N. Xi, and J. Ma, “BejaGNN: Behavior-based Java malware detection via graph neural network,” *J. Supercomput.*, vol. 79, no. 14, pp. 15 390–15 414, 2023. <https://doi.org/10.1007/s11227-023-05243-x>
- [5] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A comprehensive survey on graph neural networks,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, 2021. <https://doi.org/10.1109/TNNLS.2020.2978386>

- [6] E. Amer, S. El-Sappagh, T. Abuhamad, B. A. S. Al-Rimy, and A. Mohasseb, "GraphShield: Advanced dynamic graph-based malware detection using graph neural networks," *Expert Syst. Appl.*, vol. 298, p. 129812, 2026. <https://doi.org/10.1016/j.eswa.2025.129812>
- [7] L. Luo, J. Ming, D. Wu, P. Liu, and S. Zhu, "Semantics-based obfuscation-resilient binary code similarity comparison with applications to software and algorithm plagiarism detection," *IEEE Trans. Softw. Eng.*, vol. 43, no. 12, pp. 1157–1177, 2017. <https://doi.org/10.1109/TSE.2017.2655046>
- [8] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," *J. Netw. Comput. Appl.*, vol. 153, p. 102526, 2020. <https://doi.org/10.1016/j.jnca.2019.102526>
- [9] E. Amer and I. Zelinka, "A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence," *Comput. Secur.*, vol. 92, p. 101760, 2020. <https://doi.org/10.1016/j.cose.2020.101760>
- [10] E. Amer, I. Zelinka, and S. El-Sappagh, "A multi-perspective malware detection approach through behavioral fusion of API call sequence," *Comput. Secur.*, vol. 110, p. 102449, 2021. <https://doi.org/10.1016/j.cose.2021.102449>
- [11] C. Li, Q. Lv, N. Li, Y. Wang, D. Sun, and Y. Qiao, "A novel deep framework for dynamic malware detection based on API sequence intrinsic features," *Comput. Secur.*, vol. 116, p. 102686, 2022. <https://doi.org/10.1016/j.cose.2022.102686>
- [12] I. U. Haq, T. A. Khan, and A. Akhuzada, "A dynamic robust DL-based model for Android malware detection," *IEEE Access*, vol. 9, pp. 74 510–74 521, 2021. <https://doi.org/10.1109/ACCESS.2021.3079370>
- [13] S. R. Davies, R. Macfarlane, and W. J. Buchanan, "Differential area analysis for ransomware attack detection within mixed file datasets," *Comput. Secur.*, vol. 108, p. 102377, 2021. <https://doi.org/10.1016/j.cose.2021.102377>
- [14] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, 2022. <https://doi.org/10.1109/TDSC.2021.3051525>
- [15] R. Mahouachi, "Knowledge distillation-driven commit-aware multimodal learning for software vulnerability detection," *Autom. Softw. Eng.*, vol. 33, no. 2, p. 12, 2026. <https://doi.org/10.1007/s10515-026-00595-z>
- [16] Á. Kiss, J. Jász, and T. Gyimóthy, "Using dynamic information in the interprocedural static slicing of binary executables," *Softw. Qual. J.*, vol. 13, no. 3, pp. 227–245, 2005. <https://doi.org/10.1007/s11219-005-1751-x>
- [17] K. A. Roundy and B. P. Miller, "Binary-code obfuscations in prevalent packer tools," *ACM Comput. Surv.*, vol. 46, no. 1, pp. 1–32, 2013. <https://doi.org/10.1145/2522968.2522972>
- [18] N. Snavely, S. Debray, and G. Andrews, "Unpredication, unscheduling, unspeculation: Reverse engineering Itanium executables," *IEEE Trans. Softw. Eng.*, vol. 31, no. 2, pp. 99–115, 2005. <https://doi.org/10.1109/TSE.2005.27>
- [19] D. Kim, E. Kim, S. K. Cha, S. Son, and Y. Kim, "Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1661–1682, 2023. <https://doi.org/10.1109/TSE.2022.3187689>
- [20] V. Balachandran and S. Emmanuel, "Potent and stealthy control flow obfuscation by stack based self-modifying code," *IEEE Trans. Inf. Forensics Secur.*, vol. 8, no. 4, pp. 669–681, 2013. <https://doi.org/10.1109/TIFS.2013.2250964>
- [21] A. Afianian, S. Niksefat, B. Sadeghiyan, and D. Baptiste, "Malware dynamic analysis evasion techniques: A survey," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–28, 2019. <https://doi.org/10.1145/3365001>
- [22] M. J. Page, D. Moher, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan *et al.*, "PRISMA 2020 explanation and elaboration: Updated guidance and exemplars for reporting systematic reviews," *BMJ*, vol. 372, p. n160, 2021. <https://doi.org/10.1136/bmj.n160>
- [23] S. Ullah and H. Oh, "BinDiffNN: Learning distributed representation of assembly for robust binary diffing against semantic differences," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3442–3466, 2022. <https://doi.org/10.1109/TSE.2021.3093926>
- [24] M. Amouei, B. C. M. Fung, P. Charland, and J. Meng, "LENA: Llama-based embeddings of neutralized assembly for cross-compiler/optimization binary code similarity detection," *IEEE Trans. Softw. Eng.*, pp. 1–24, 2026. <https://doi.org/10.1109/TSE.2026.3705321>
- [25] S. Jiang, C. Fu, S. He, J. Lv, L. Han, and H. Hu, "BinCola: Diversity-sensitive contrastive learning for binary code similarity detection," *IEEE Trans. Softw. Eng.*, vol. 50, no. 10, pp. 2485–2497, 2024. <https://doi.org/10.1109/TSE.2024.3411072>
- [26] N. Ito, M. Hashimoto, and A. Otsuka, "Feature extraction methods for binary code similarity detection using neural machine translation models," *IEEE Access*, vol. 11, pp. 102 796–102 805, 2023. <https://doi.org/10.1109/ACCESS.2023.3316215>

- [27] A. Sabbah, R. Jarrar, S. Zein, and D. Mohaisen, “Understanding concept drift with deprecated permissions in Android malware detection,” *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 4, pp. 8292–8306, 2026. <https://doi.org/10.1109/TDSC.2026.3682216>
- [28] A. Patel, D. S. Tomar, R. Pateriya, and R. HariPriya, “Mitigating concept drift in malware classification: A temporal and resource-aware evaluation,” *Knowl.-Based Syst.*, vol. 350, p. 116534, 2026. <https://doi.org/10.1016/j.knosys.2026.116534>
- [29] G. Apruzzese, L. Pajola, and M. Conti, “The cross-evaluation of machine learning-based network intrusion detection systems,” *IEEE Trans. Netw. Serv. Manag.*, vol. 19, no. 4, pp. 5152–5169, 2022. <https://doi.org/10.1109/TNSM.2022.3157344>
- [30] G. Liu, X. Zhou, J. Pang, F. Yue, W. Liu, and J. Wang, “Codeformer: A GNN-nested transformer model for binary code similarity detection,” *Electronics*, vol. 12, no. 7, p. 1722, 2023. <https://doi.org/10.3390/electronics12071722>
- [31] G. Brauwers and F. Frasincar, “A general survey on attention mechanisms in deep learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3279–3298, 2023. <https://doi.org/10.1109/TKDE.2021.3126456>
- [32] C. Wang and D. Miyazaki, “PolAttNet: A deep learning framework for polarized image reflection removal with multi-attention mechanisms,” *IEEE Access*, vol. 13, pp. 30 762–30 778, 2025. <https://doi.org/10.1109/ACCESS.2025.3541640>
- [33] P. H. Le-Khac, G. Healy, and A. F. Smeaton, “Contrastive representation learning: A framework and review,” *IEEE Access*, vol. 8, pp. 193 907–193 934, 2020. <https://doi.org/10.1109/ACCESS.2020.3031549>
- [34] J. Zhu, Y. Xia, L. Wu, J. Deng, W. Zhou, T. Qin, T. Y. Liu, and H. Li, “Masked contrastive representation learning for reinforcement learning,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3421–3433, 2023. <https://doi.org/10.1109/TPAMI.2022.3176413>
- [35] Y. Lin, Y. Gou, X. Liu, J. Bai, J. Lv, and X. Peng, “Dual contrastive prediction for incomplete multi-view representation learning,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 1–14, 2022. <https://doi.org/10.1109/TPAMI.2022.3197238>
- [36] E. Vaaras, M. Airaksinen, and O. Räsänen, “PFML: Self-supervised learning of time-series data without representation collapse,” *IEEE Access*, vol. 13, pp. 60 233–60 244, 2025. <https://doi.org/10.1109/ACCESS.2025.3556957>
- [37] S. Liu, D. Cao, C. Chen, Z. Li, J. Zhang, S. Camtepe, and Y. Xiang, “Cross-architecture software vulnerability analysis in binary code,” *IEEE Trans. Inf. Forensics Secur.*, vol. 21, pp. 7752–7766, 2026. <https://doi.org/10.1109/TIFS.2026.3723092>

Nomenclature

X_O	Opcode instruction sequence modality ($X_O \in \mathbb{R}^{L \times d_{\text{vocab}}}$)
G_C	Control-flow graph topology modality ($G_C = (V, E)$)
S_D	Dynamic system-call execution trace modality ($S_D = (s_1, \dots, s_T)$)
h_m	Modality-specific representation vector for modality $m \in \{O, C, S\}$
z_m	Normalized unit hypersphere projection vector ($z_m \in \mathbb{S}^{d-1}$)
z_{fused}	Reliability-weighted fused multimodal embedding vector ($z_{\text{fused}} \in \mathbb{R}^d$)
α_m	Reliability-aware modality fusion gating weight for modality m ($\sum_m \alpha_m = 1$)
r_m	Dynamic modality reliability confidence score for modality m ($r_m \in [0, 1]$)
M_m	Binary modality availability indicator mask ($M_m \in \{0, 1\}$)
τ	Temperature hyperparameter in InfoNCE contrastive objective ($\tau = 0.07$)
$\mathcal{L}_{\text{InfoNCE}}$	Multi-channel Information Noise Contrastive Estimation objective loss
$\mathcal{L}_{\text{Gram}}$	Volumetric Gram determinant representation regularizer
$\mathcal{L}_{\text{Rel}}$	Reliability-aware corruption and modality dropout loss
$\mathcal{L}_{\text{Total}}$	Multi-task objective loss unifying task, alignment, and regularization